

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART VII STORAGE

CSI, Distributed Storage, AI Artifacts, Backup and Disaster Recovery

Volume 7 of 16 Core Series

Prerequisites: Parts I through VI

Edition 2025-2026

TABLE OF CONTENTS

1. Kubernetes Storage Architecture	3
2. CSI Architecture and Driver Development	7
3. Dynamic Provisioning Deep Dive	12
4. Volume Snapshots and Cloning	16
5. Cloud Storage Integrations (EBS, GCS, Azure)	19
6. Distributed Storage: Ceph / Rook	23
7. Longhorn -- Cloud-Native Block Storage	28
8. OpenEBS -- Container-Attached Storage	32
9. NFS and Shared Filesystems	35
10. Storage Performance Optimisation	38
11. AI Artifact Storage Patterns	42
12. Model Repository Architecture	46
13. Vector Database Storage	50
14. Backup and Disaster Recovery	53
15. Storage Observability	58
16. Storage Anti-Patterns	61
17. Hands-On Exercises	64

CHAPTER 1

Kubernetes Storage Architecture

Kubernetes storage is a three-layer abstraction designed to decouple storage provisioning (an infrastructure concern) from storage consumption (an application concern). Understanding each layer is essential for designing storage systems that are performant, durable, and operationally manageable at enterprise scale.

The Three Storage Layers

- * **StorageClass (infrastructure layer):** Defines HOW storage is provisioned. Contains the provisioner name, parameters (disk type, IOPS, encryption), reclaim policy, and binding mode. Created by the storage administrator. Analogous to a product catalog entry for storage types.
- * **PersistentVolume (storage layer):** Represents an actual piece of provisioned storage. Can be statically created (pre-provisioned) or dynamically created by a CSI provisioner. Lives at the cluster scope -- not namespaced. Contains the actual volume details (volume ID, capacity, access modes, reclaim policy).
- * **PersistentVolumeClaim (workload layer):** A request for storage by an application. Created by developers; specifies desired capacity, access modes, and StorageClass. The PVC controller binds a PVC to an appropriate PV, either existing or newly provisioned. PVCs are namespaced.

Storage Lifecycle

STATIC PROVISIONING: Admin creates PV manually -> Dev creates PVC -> Binder matches PVC to PV
DYNAMIC PROVISIONING (recommended): Dev creates PVC (with StorageClass) -> PVC Controller detects unbound PVC -> CSI Provisioner creates volume in backend -> PV auto-created and bound
VOLUME BINDING MODES: Immediate: PV created/bound when PVC created Problem: PV may be in wrong zone for Pod WaitForFirstConsumer: PV created when Pod is scheduled Ensures volume is in same zone as Pod RECOMMENDED for zonal storage (EBS, Azure Disk)
RECLAIM POLICIES: Delete: PV and underlying volume deleted when PVC deleted Use for ephemeral/dev storage Retain: PV and volume kept; must be manually cleaned up REQUIRED for production databases Recycle: Deprecated; do not use

Storage Class Design Matrix

StorageClass	Backend	Access Mode	IOPS	Use Case
ultra-ssd	AWS io2/GCP pd-extreme	RWO	64,000	Databases, etcd
fast-ssd	AWS gp3/GCP pd-ssd	RWO	16,000	App storage, AI checkpoints
standard	AWS gp2/GCP pd-standard	RWO	3,000	General workloads
shared-nfs	NFS/NetApp	RWX	Variable	Shared datasets, config
fast-shared	Lustre/WekaFS	RWX	1,000,000+	AI training datasets

StorageClass	Backend	Access Mode	IOPS	Use Case
object-backed	Rook-Ceph RGW	RWX	Variable	Model artifacts, logs
local-nvme	Local NVMe (node-local)	RWO	500,000+	Ultra-low latency DB

CHAPTER 2

CSI Architecture and Driver Development

The Container Storage Interface (CSI) is the standard API for storage plugins in Kubernetes. It replaced the in-tree volume plugins (which required Kubernetes core code changes) with an out-of-tree plugin model where storage vendors implement a gRPC server that Kubernetes calls.

CSI Architecture

```
CSI driver components (typically deployed as DaemonSet + Deployment): CONTROLLER PLUGIN (Deployment -- runs anywhere): Implements: ControllerService Operations: CreateVolume, DeleteVolume, ControllerPublishVolume (attach/detach volume to/from node) Sidecars: external-provisioner, external-attacher, external-snapshotter NODE PLUGIN (DaemonSet -- runs on every node): Implements: NodeService Operations: NodePublishVolume (mount to Pod), NodeUnpublishVolume (unmount from Pod) NodeStageVolume (global mount on node -- for block devices) Sidecar: node-driver-registrar CSI RPC FLOW for dynamic provisioning: 1. Dev creates PVC 2. external-provisioner sidecar watches PVC 3. external-provisioner calls CSI.CreateVolume RPC 4. CSI driver creates volume in backend (AWS CreateVolume API, etc.) 5. CSI returns volume_id 6. external-provisioner creates PV with volume_id 7. PV bound to PVC 8. Pod scheduled 9. external-attacher calls CSI.ControllerPublishVolume (attaches EBS volume to EC2 instance) 10. kubelet calls CSI.NodePublishVolume (mounts /dev/xvdf into Pod filesystem)
```

CSI Sidecar Containers

Sidecar	Role	Watches
external-provisioner	Creates/deletes volumes; creates/deletes PVs	PVC with matching StorageClass
external-attacher	Calls ControllerPublish/Unpublish Volume	VolumeAttachment objects
external-snapshotter	Creates/deletes volume snapshots	VolumeSnapshot objects
external-resizer	Expands volumes	PVC with increased storage request
node-driver-registrar	Registers CSI driver with kubelet	NodeServer socket
livenessprobe	Health check for CSI driver	CSI driver gRPC endpoint

CSI Driver Capabilities Matrix

Driver	Provisioning	Snapshots	Resize	RWX	Encryption	Clone
aws-ebs-csi	Yes	Yes	Yes	No	KMS	Yes
gcp-pd-csi	Yes	Yes	Yes	No	CMEK	Yes
azure-disk-csi	Yes	Yes	Yes	No	CMEK	Yes

Driver	Provisioning	Snapshots	Resize	RWX	Encryption	Clone
azure-file-csi	Yes	No	Yes	Yes	No	No
efs-csi (AWS)	Yes	No	No	Yes	KMS	No
rook-ceph-rbd	Yes	Yes	Yes	No	LUKS	Yes
rook-ceph-cephfs	Yes	Yes	Yes	Yes	LUKS	No
longhorn	Yes	Yes	Yes	No	Yes	Yes
local-path-provisioner	Yes	No	No	No	No	No

CHAPTER 3

Dynamic Provisioning Deep Dive

Dynamic provisioning automates the creation of PersistentVolumes when a PVC is submitted. This eliminates the need for storage admins to pre-provision volumes, enabling self-service storage for development teams.

StorageClass Production Examples

```
# AWS EBS gp3 with encryption: apiVersion: storage.k8s.io/v1 kind: StorageClass metadata:
name: ebs-gp3-encrypted annotations: storageclass.kubernetes.io/is-default-class: 'true'
provisioner: ebs.csi.aws.com parameters: type: gp3 iops: '3000' throughput: '125' encrypted:
'true' kmsKeyId: arn:aws:kms:us-east-1:123456789:key/mrk-abc123 reclaimPolicy: Retain #
CRITICAL for production data volumeBindingMode: WaitForFirstConsumer allowVolumeExpansion:
true mountOptions: - noatime - nodiratime # GCP SSD persistent disk: apiVersion:
storage.k8s.io/v1 kind: StorageClass metadata: name: gcp-ssd provisioner:
pd.csi.storage.gke.io parameters: type: pd-ssd disk-encryption-kms-key:
projects/myproject/locations/us-central1/keyRings/myring/cryptoKeys/mykey reclaimPolicy:
Retain volumeBindingMode: WaitForFirstConsumer allowVolumeExpansion: true # Local NVMe for
maximum performance: apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name:
local-nvme provisioner: kubernetes.io/no-provisioner volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete # Note: local volumes require manual PV creation per node
```

PVC Best Practices

```
apiVersion: v1 kind: PersistentVolumeClaim metadata: name: database-data namespace: production
labels: app: postgres tier: database annotations: # Document PVC purpose and owner:
storage.company.com/owner: 'platform-team' storage.company.com/backup: 'daily' spec:
accessModes: [ReadWriteOnce] storageClassName: ebs-gp3-encrypted resources: requests: storage:
100Gi # Volume expansion (after initial creation, only increase allowed): # Edit
resources.requests.storage to larger value # StorageClass must have allowVolumeExpansion: true
# Production checklist for PVCs: # 1. Always use Retain reclaim policy for production
databases # 2. Use WaitForFirstConsumer binding mode for zonal storage # 3. Enable volume
expansion in StorageClass # 4. Size 20-30% larger than current need # 5. Monitor usage with
kubelet_volume_stats_used_bytes metric
```


CHAPTER 4

Volume Snapshots and Cloning

Volume snapshots enable point-in-time copies of PersistentVolumes for backup, testing, and disaster recovery. Volume cloning creates a new volume pre-populated with the data from an existing PVC. Both are implemented via CSI and require driver support.

Volume Snapshot Workflow

```
# 1. Define a VolumeSnapshotClass: apiVersion: snapshot.storage.k8s.io/v1 kind:
VolumeSnapshotClass metadata: name: ebs-vsc annotations:
snapshot.storage.kubernetes.io/is-default-class: 'true' driver: ebs.csi.aws.com
deletionPolicy: Retain # Keep snapshot even if VolumeSnapshot deleted parameters:
tagSpecification_1: 'key=backup-type,value=pre-upgrade' # 2. Take a snapshot: apiVersion:
snapshot.storage.k8s.io/v1 kind: VolumeSnapshot metadata: name: postgres-backup-20250601
namespace: production spec: volumeSnapshotClassName: ebs-vsc source:
persistentVolumeClaimName: database-data # 3. Check snapshot readiness: kubectl get
volumesnapshot postgres-backup-20250601 -n production kubectl describe volumesnapshot
postgres-backup-20250601 -n production # Wait for: ReadyToUse: true # 4. Restore from snapshot
(new PVC from snapshot): apiVersion: v1 kind: PersistentVolumeClaim metadata: name:
database-restored namespace: production spec: storageClassName: ebs-gp3-encrypted dataSource:
name: postgres-backup-20250601 kind: VolumeSnapshot apiGroup: snapshot.storage.k8s.io
accessModes: [ReadWriteOnce] resources: requests: storage: 100Gi
```

Volume Cloning

```
# Clone a PVC to a new PVC (pre-populated with source data): apiVersion: v1 kind:
PersistentVolumeClaim metadata: name: database-clone-for-testing namespace: staging spec:
storageClassName: ebs-gp3-encrypted dataSource: name: database-data # Source PVC (must be in
same namespace) kind: PersistentVolumeClaim accessModes: [ReadWriteOnce] resources: requests:
storage: 100Gi # Use case: Copy production DB to staging for testing # Much faster than
backup/restore; no data movement at storage layer # Storage efficiency: copy-on-write at
backend (only changes stored)
```

CHAPTER 5

Cloud Storage Integrations

Cloud	Block Storage	File Storage	Object Storage	Key Considerations
AWS	EBS (gp3, io2) via aws-ebs-csi	EFS via aws-efs-csi	S3 via Mountpoint	EBS = zonal; EFS = regional; io2 for IOPS-heavy
GCP	Persistent Disk via pd-csi	Filestore via filestore-csi	GCS via gcsfuse	pd-extreme for ultra-high IOPS; balanced for general
Azure	Managed Disk via azuredisk-csi	Azure Files via azurefile-csi	Blob via blobfuse2	Ultra Disk for VMs with UltraSSD support enabled
On-prem	Ceph RBD, vSphere VMDK	CephFS, NFS, NetApp	MinIO, Ceph RGW	Ceph = unified; NetApp = enterprise support + snapshots

AWS EBS CSI -- Production Configuration

```
# Install AWS EBS CSI Driver (IAM required): helm repo add aws-ebs-csi-driver
https://kubernetes-sigs.github.io/aws-ebs-csi-driver helm install aws-ebs-csi-driver
aws-ebs-csi-driver/aws-ebs-csi-driver \ --namespace kube-system \ --set
controller.serviceAccount.annotations.\
eks\.amazonaws\.com/role-arn=arn:aws:iam::ACCOUNT:role/EBSCSIRole \ --set
enableVolumeScheduling=true \ --set enableVolumeResizing=true \ --set
enableVolumeSnapshot=true # StorageClass tiers: # gp3 base: 3000 IOPS, 125 MB/s,
$0.08/GB-month # gp3 tuned: up to 16000 IOPS, 1000 MB/s (+$0.006/IOPS above 3000) # io2: up to
64000 IOPS, $0.125/GB + $0.065/IOPS # Cost optimisation: use gp3 not gp2 # gp2: 3 IOPS/GB
baseline (100GB = 300 IOPS) # gp3: always 3000 IOPS baseline (100GB = 3000 IOPS) # gp3 is 20%
cheaper and 10x more performant by default
```

CHAPTER 6

Distributed Storage: Ceph / Rook

Ceph is the most widely deployed open-source distributed storage system. It provides block (RBD), file (CephFS), and object (RADOS Gateway) storage from a single storage cluster. Rook is the Kubernetes Operator for Ceph, making it possible to run and manage Ceph entirely within Kubernetes.

Ceph Architecture

```
Ceph components: MON (Monitors): 3 or 5; maintain cluster map using Paxos consensus MGR
(Managers): 2; metrics, dashboards, orchestration interface OSD (Object Storage Daemons): 1
per disk; store data, handle replication MDS (Metadata Servers): for CephFS only; manage
filesystem namespace RGW (RADOS Gateway): S3/Swift compatible object storage endpoint Data
distribution (CRUSH algorithm): Data is divided into objects -> placed in placement groups
(PGs) CRUSH map determines which OSDs store each PG Default: 3-way replication (data stored on
3 different OSDs) Alternative: erasure coding (k+m coding; more efficient, higher latency)
Performance characteristics: Latency: 0.5-5ms (NVMe-backed); 5-20ms (HDD-backed) Throughput:
Linear with OSD count; 1GB/s per NVMe OSD typical IOPS: 100K+ IOPS per NVMe OSD cluster Scale:
Petabytes; thousands of OSDs # Rook-Ceph quick start: kubectl apply -f
https://raw.githubusercontent.com/rook/rook/master/deploy/examples/crds.yaml kubectl apply -f
https://raw.githubusercontent.com/rook/rook/master/deploy/examples/common.yaml kubectl apply
-f https://raw.githubusercontent.com/rook/rook/master/deploy/examples/operator.yaml
```

Rook CephCluster Configuration

```
apiVersion: ceph.rook.io/v1 kind: CephCluster metadata: name: rook-ceph namespace: rook-ceph
spec: cephVersion: image: quay.io/ceph/ceph:v18.2 dataDirHostPath: /var/lib/rook mon: count: 3
allowMultiplePerNode: false mgr: count: 2 modules: - name: pg_autoscaler enabled: true - name:
dashboard enabled: true dashboard: enabled: true ssl: true storage: useAllNodes: false
useAllDevices: false nodes: - name: storage-node-01 devices: - name: nvme0n1 - name: nvme1n1 -
name: storage-node-02 devices: - name: nvme0n1 network: provider: host # Host networking for
maximum performance selectors: public: en01 # Separate public and cluster networks cluster:
en02 placement: all: tolerations: - key: storage-node operator: Exists effect: NoSchedule
```

CHAPTER 7

Longhorn -- Cloud-Native Block Storage

Longhorn (CNCF incubating, from Rancher/SUSE) is a lightweight, distributed block storage system for Kubernetes. Unlike Ceph (which requires dedicated storage nodes), Longhorn uses the existing worker node disks, making it ideal for smaller clusters, edge deployments, and environments without dedicated storage infrastructure.

Longhorn Architecture

```
Longhorn storage architecture: Longhorn Manager (DaemonSet on every node): Manages volume lifecycle on the node Schedules replicas across nodes Longhorn Engine (per volume, per node): Frontend: exposes block device to Pod Replication: syncs data to replica processes on other nodes Replica (per volume, per node): Stores actual data on node-local disk Each volume has N replicas (default: 3) Volume replication: Write path: Pod -> Engine -> (replica-1, replica-2, replica-3) All replicas must acknowledge before write confirmed Node failure: Longhorn automatically rebuilds missing replica Features: * CSI compliant (block only) * Volume snapshots and backups to S3/NFS * Recurring backup schedules * Volume encryption (LUKS) * RWO only (block storage) * Live volume expansion * Disaster recovery volumes (cross-cluster backup/restore)
```

Longhorn vs Ceph Decision Matrix

Dimension	Longhorn	Ceph/Rook
Deployment complexity	Low (helm install)	High (dedicated nodes, tuning)
Minimum cluster size	3 nodes	5+ nodes recommended
Dedicated storage nodes	No (uses worker disks)	Yes (recommended)
Storage types	Block (RWO) only	Block, File (RWX), Object
Performance	Good (network replication overhead)	Excellent (NVMe-backed)
Snapshots	Yes (software snapshots)	Yes (RBD snapshots, very fast)
Backup	Yes (to S3, NFS)	Yes (RBD mirror, RGW)
Scale	Small-medium (up to 500TB)	Petabyte scale
UI	Built-in web UI	Ceph Dashboard, custom
Best for	Edge, small clusters, RKE/k3s	Large enterprise, AI storage

CHAPTER 8

OpenEBS -- Container-Attached Storage

OpenEBS (CNCF sandbox) pioneered the Container-Attached Storage (CAS) pattern: each volume has its own dedicated storage controller running as a Pod. This gives each volume complete isolation -- a noisy volume cannot impact others. OpenEBS provides multiple storage engines for different use cases.

Engine	Type	Technology	Use Case
Mayastor (now OpenEBS v3)	Block	NVMe-oF/SPDK	Ultra-high performance (sub-ms latency)
LocalPV-Hostpath	Block	Hostpath bind mount	Simple local storage; no replication
LocalPV-ZFS	Block	ZFS on node	Snapshots, compression, checksums locally
LocalPV-LVM	Block	LVM on node	Volume groups, thin provisioning
Jiva	Block	iSCSI	Legacy; use Mayastor instead

Mayastor (OpenEBS v3) -- NVMe Performance

Mayastor uses SPDK (Storage Performance Development Kit) and NVMe-oF to deliver sub-millisecond latency and near-wire-speed throughput for Kubernetes volumes. It is purpose-built for performance-sensitive AI and database workloads:

```
Mayastor performance characteristics: Latency: 100-500 microseconds (vs 1-5ms for traditional CSI) IOPS: 1M+ IOPS (limited by NVMe hardware) Throughput: Near line-rate on 100Gbps NICs CPU: SPDK poll mode (dedicated cores) for zero-copy I/O # Install Mayastor: helm repo add openebs https://openebs.github.io/charts helm install openebs openebs/openebs \ --namespace openebs --create-namespace \ --set mayastor.enabled=true \ --set localprovisioner.enabled=true # Node requirements for Mayastor: # hugepages-2Mi: 2Gi # SPDK requires huge pages # Dedicated CPU cores for SPDK reactor threads
```

CHAPTER 9

NFS and Shared Filesystems

NFS (Network File System) provides ReadWriteMany (RWX) access -- multiple Pods on multiple nodes mounting the same filesystem simultaneously. This is essential for shared datasets in AI training, shared configuration, and legacy applications requiring shared filesystem semantics.

NFS-based StorageClass with nfs-subdir-external-provisioner

```
# Install NFS provisioner: helm repo add nfs-subdir-external-provisioner \
https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner/ helm install
nfs-provisioner \ nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \ --set
nfs.server=nfs.internal.corp \ --set nfs.path=/shared/k8s-volumes \ --set
storageClass.name=nfs-shared \ --set storageClass.reclaimPolicy=Retain # PVC using NFS:
apiVersion: v1 kind: PersistentVolumeClaim metadata: name: shared-dataset spec: accessModes:
[ReadWriteMany] storageClassName: nfs-shared resources: requests: storage: 10Ti # Mount same
PVC in multiple Pods simultaneously: # Training workers read dataset concurrently #
Preprocessing jobs write data while training reads it
```

High-Performance Parallel Filesystems for AI

Filesystem	Protocol	Peak Throughput	Latency	K8s Integration
Lustre	Custom	TB/s aggregate	Low ms	Lustre CSI (Trident, Weka)
GPFS/IBM Spectrum Scale	NFS/GPFS	500GB/s+	Sub-ms	Spectrum Scale CSI
WekaFS	WekaFS+NFS	Up to 100GB/s per client	Sub-ms	Weka CSI
BeeGFS	BeeGFS	High; scales linearly	Low ms	BeeGFS CSI (ThinkParQ)
Quobyte	Quobyte protocol	High	Low ms	Quobyte CSI

CHAPTER 10

Storage Performance Optimisation

Storage is frequently the bottleneck in both database and AI workloads. Optimising storage performance in Kubernetes requires understanding the full I/O path: application -> kernel page cache -> filesystem -> block device -> CSI driver -> network (for remote storage) -> backend storage.

I/O Path Optimisation

```
# Kernel page cache tuning (on node, via DaemonSet sysctl): vm.dirty_ratio = 15 # % RAM for
dirty pages before sync vm.dirty_background_ratio = 5 # % RAM for background dirty writeback
vm.dirty_writeback_centisecs = 500 # Writeback every 5 seconds # For databases (disable page
cache, use O_DIRECT): # PostgreSQL: already uses O_DIRECT for WAL # fsync: always enabled in
production (never disable) # Block device queue depth (more parallel I/Os): echo mq-deadline >
/sys/block/nvme0n1/queue/scheduler echo 64 > /sys/block/nvme0n1/queue/nr_requests # XFS mount
options for database workloads: mount -o noatime,nodiratime,nobarrier,inode64,allocsize=64m
/dev/xvdf /data # noatime: no access time updates (reduces write I/O) # nobarrier: disable
write barriers (only safe with battery-backed cache) # allocsize: large allocation unit for
sequential workloads
```

Storage Performance Benchmarking

```
# Run fio inside a Pod to benchmark PVC performance: kubectl run fio-bench --image=ljishen/fio
--restart=Never -it \ --overrides='{ "spec": { "containers": [ { "name": "fio", "image": "ljishen/fio",
"volumeMounts": [ { "name": "bench", "mountPath": "/data" } ], "resources": { "requests": { "cpu": "4", "memo
ry": "4Gi" } } } ], "volumes": [ { "name": "bench", "persistentVolumeClaim": { "claimName": "bench-pvc" } } ] } }'
# Sequential read (streaming throughput for AI training): fio --name=seq-read --rw=read
--bs=1M --numjobs=4 --size=10G \ --directory=/data --runtime=60 --time_based --group_reporting
# Random read (OLTP database pattern): fio --name=rand-read --rw=randread --bs=4K --numjobs=16
\ --size=10G --directory=/data --runtime=60 --time_based # Sequential write (checkpoint,
backup): fio --name=seq-write --rw=write --bs=1M --numjobs=4 --size=10G \ --directory=/data
--runtime=60 --time_based --fsync=1
```

CHAPTER 11

AI Artifact Storage Patterns

AI and ML workloads have storage requirements fundamentally different from traditional enterprise applications. The size, access pattern, lifecycle, and sharing requirements of AI artifacts demand specific storage architectures for each artifact type.

AI Artifact Storage Requirements

Artifact Type	Typical Size	Access Pattern	Sharing	Recommended Storage
LLM Weights (full)	7GB - 700GB	Read-heavy, sequential	Multi-pod RO	Object store + RWX PVC (WekaFS/NFS)
Fine-tuned adapter (LoRA)	10MB - 5GB	Read-heavy	Multi-pod RO	Object store or RWO PVC
Training dataset	100GB - 100TB	Sequential read, parallel	Multi-pod RO	Parallel FS (Lustre, WekaFS) or S3
Training checkpoint	1GB - 50GB	Write-heavy (training), read (resume)	Single writer	RWO NVMe SSD PVC
Embeddings/Index	1GB - 10TB	Random read, write	Vector DB Pod	NVMe SSD RWO PVC
Experiment artifacts	Variable	Write once, read occasionally	Single namespace	Object store (MLflow, W&B;)
Feature store data	1GB - 1TB	Random read, batch write	Multi-service	Redis/Feast on SSD PVC
ONNX/TensorRT model	100MB - 10GB	Read-only serving	Multi-replica RO	Container image or object store

Model Weight Serving Pattern

Serving large models (70B+ parameters) requires efficient weight loading. Three patterns are used in production:

- * **Object store streaming:** Models stored in S3/GCS; KServe or vLLM streams weights directly from object store on startup. Simplest operational model; startup latency proportional to model size (100GB at 10GB/s = 10s).
- * **Shared RWX PVC:** Model weights stored on shared NFS/parallel filesystem PVC; multiple serving replicas mount same PVC read-only. Fast access after first load (no re-download); good for models served by multiple replicas.

- * **Model baked into container layer:** Small models (under 2GB) can be embedded in the container image. Pull-through registry cache ensures fast node-local loading. Simplest for small models; impractical for large LLMs.

S3-Compatible Model Storage with Mountpoint

```
# Mount S3 bucket as a PVC using Mountpoint for Amazon S3 CSI: # (Also works with MinIO for
on-premises S3-compatible storage) apiVersion: v1 kind: PersistentVolume metadata: name:
model-weights-pv spec: capacity: storage: 10Ti accessModes: [ReadOnlyMany] mountOptions: -
allow-other - read-only csi: driver: s3.csi.aws.com volumeHandle: s3-model-weights-vol
volumeAttributes: bucketName: company-model-registry region: us-east-1 --- # Use in inference
serving Pod: volumes: - name: model-weights persistentVolumeClaim: claimName:
model-weights-pvc readOnly: true containers: - name: vllm volumeMounts: - name: model-weights
mountPath: /models env: - name: MODEL_PATH value: /models/llama-3-70b
```

CHAPTER 12

Model Repository Architecture

A model repository (model registry) is the system of record for ML models: it tracks model versions, metadata, lineage, metrics, and deployment status. In Kubernetes, the model registry must integrate with CI/CD, feature stores, inference serving, and observability systems.

Model Registry Architecture Patterns

Registry	Backend Storage	K8s Integration	Features
MLflow	S3, Azure Blob, local	Tracking server as Deployment; models from S3	Tracking, metrics, packaging, registry
Weights & Biases	W&B; cloud	SDK integration; artifacts from W&B;	Experiment tracking, sweeps, artifacts
Kubeflow Model Registry	PostgreSQL + object store	CRD-based; native K8s	K8s-native versioning, serving integration
BentoML + Yatai	Object store + PostgreSQL	Yatai as K8s operator	Bento building, registry, deployment
Seldon Core v2	OCI registry	CRD-based model server	Model mesh, A/B testing, explainability
DVC (Data Version Control)	Git + S3/GCS	Git-based; S3 for large files	Dataset + model versioning with Git

MLflow Deployment on Kubernetes

```
apiVersion: apps/v1 kind: Deployment metadata: name: mlflow-tracking namespace: mlops spec: replicas: 2 selector: matchLabels: { app: mlflow } template: spec: containers: - name: mlflow image: ghcr.io/mlflow/mlflow:v2.13 command: - mlflow - server - --backend-store-uri=postgresql://mlflow:PASSWORD@postgres:5432/mlflow - --default-artifact-root=s3://company-mlflow-artifacts/ - --host=0.0.0.0 - --port=5000 - --workers=4 env: - name: AWS_DEFAULT_REGION value: us-east-1 - name: MLFLOW_S3_ENDPOINT_URL value: https://minio.internal.corp resources: requests: { cpu: 500m, memory: 1Gi } limits: { memory: 2Gi }
```

CHAPTER 13

Vector Database Storage

Vector databases are central to RAG (Retrieval-Augmented Generation) architectures. Their storage requirements are distinct: they maintain large in-memory indexes (for ANN search), persistent storage for durability, and require careful PVC sizing and performance tuning for production deployments.

Vector Database Storage Sizing

Database	Index Type	Memory Requirement	Disk Requirement	RWX Support
Weaviate	HNSW (in-memory)	1.5x vector size in RAM	3x for persistence	No (StatefulSet)
Qdrant	HNSW + mmap	Configurable (on-disk mode)	2x vector size	No (StatefulSet)
Milvus	Multiple (HNSW, IVF, FLAT)	Variable by index type	5x for segments	No (sharded)
pgvector	B-tree + IVFFlat	PostgreSQL shared_buffers	WAL + data files	No (StatefulSet)
Chroma	HNSW	In-process (embedding size)	SQLite + parquet	No
Pinecone	Managed SaaS	N/A (managed)	N/A (managed)	N/A

Qdrant on Kubernetes -- Production Deployment

```
# Qdrant StatefulSet with NVMe SSD storage: apiVersion: apps/v1 kind: StatefulSet metadata:
name: qdrant namespace: ai-platform spec: serviceName: qdrant-headless replicas: 3 selector:
matchLabels: { app: qdrant } template: spec: containers: - name: qdrant image:
qdrant/qdrant:v1.9.0 ports: - { name: http, containerPort: 6333 } - { name: grpc,
containerPort: 6334 } - { name: p2p, containerPort: 6335 } resources: requests: cpu: '4'
memory: 32Gi limits: memory: 64Gi env: - name: QDRANT__CLUSTER__ENABLED value: 'true' - name:
QDRANT__STORAGE__STORAGE_PATH value: /qdrant/storage volumeMounts: - name: storage mountPath:
/qdrant/storage volumeClaimTemplates: - metadata: name: storage spec: accessModes:
[ReadWriteOnce] storageClassName: fast-nvme resources: requests: storage: 500Gi
```

CHAPTER 14

Backup and Disaster Recovery

Kubernetes backup encompasses two distinct concerns: application data (PVC contents) and cluster state (etcd). A complete DR strategy must address both, with clearly defined RPO (Recovery Point Objective) and RTO (Recovery Time Objective) targets.

Velero -- Kubernetes Backup and DR

Velero is the de facto standard for Kubernetes backup. It backs up Kubernetes resources (as YAML) and PersistentVolume data (via CSI snapshots or Restic/Kopia file-level backup) to object storage.

```
# Install Velero (AWS S3 backend): velero install \ --provider aws \ --plugins
velero/velero-plugin-for-aws:v1.9.0 \ --bucket my-velero-backups \ --backup-location-config
region=us-east-1 \ --snapshot-location-config region=us-east-1 \ --secret-file
./credentials-velero \ --use-node-agent \ --features=EnableCSI # Create a scheduled backup
(daily at 2AM UTC): velero schedule create daily-backup \ --schedule='0 2 * * *' \
--include-namespaces production,staging \ --ttl 720h \ --snapshot-volumes=true # Create
on-demand backup: velero backup create pre-upgrade-backup \ --include-namespaces production \
--snapshot-volumes=true \ --wait # Restore from backup: velero restore create --from-backup
pre-upgrade-backup \ --include-namespaces production \ --namespace-mappings
production:production-restored # Check backup status: velero backup describe
pre-upgrade-backup velero backup logs pre-upgrade-backup
```

DR Architecture Patterns

Pattern	RTO	RPO	Cost	Implementation
Active-Passive (warm)	15-60 min	Minutes (async replication)	Medium	Velero backups + DR cluster in warm standby
Active-Active (multi-region)	Seconds	Near-zero	High	Cilium Cluster Mesh + global load balancer
Backup-Restore	Hours	Hours (backup frequency)	Low	Velero scheduled backup to S3
Pilot Light	30-60 min	Minutes	Low-Medium	Minimal DR cluster + data replication

CHAPTER 15

Storage Observability

Key Storage Metrics

Metric	Source	Alert Threshold	Action
kubelet_volume_stats_used_bytes / capacity_bytes	kubelet	Greater than 80% full	Expand PVC or add storage
kubelet_volume_stats_inodes_used / total	kubelet	Greater than 80%	Clean up small files; expand
node_disk_io_time_seconds_total	node_exporter	Greater than 80% util	Move to faster storage tier
node_disk_read_bytes_total	node_exporter	Near line rate	Investigate hot spot
node_disk_write_bytes_total	node_exporter	Near line rate	Check for runaway write processes
ceph_health_status	rook-ceph	Not 0 (HEALTH_OK)	Investigate Ceph cluster health
longhorn_volume_actual_size_bytes	longhorn	Near PVC capacity	Expand volume

Storage Capacity Planning

```
# Prometheus queries for storage capacity planning: # PVCs approaching capacity (> 80% full):
kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes > 0.8 # Fastest growing
PVCs (will fill in < 7 days): predict_linear(kubelet_volume_stats_used_bytes[6h], 7*24*3600) >
kubelet_volume_stats_capacity_bytes # Node disk I/O saturation:
rate(node_disk_io_time_seconds_total[5m]) > 0.8 # Storage throughput by PVC: sum by
(persistentvolumeclaim) ( rate(kubelet_volume_stats_used_bytes[5m]) ) # Velero backup success
rate: velero_backup_success_total / velero_backup_attempt_total < 1
```

CHAPTER 16

Storage Anti-Patterns

Anti-Pattern: Using Delete reclaim policy for production databases

Problem: PVC and underlying volume deleted when StatefulSet is accidentally deleted. Complete data loss.

Solution: Set reclaimPolicy: Retain in StorageClass. Enforce via OPA/Kyverno admission policy.

Anti-Pattern: Immediate volumeBindingMode with zonal storage

Problem: PVC bound to volume in wrong availability zone; Pod cannot schedule near its data.

Solution: Always use WaitForFirstConsumer for EBS, Azure Disk, GCP PD. Only Immediate for cluster-wide storage.

Anti-Pattern: Not sizing headroom

Problem: PVC fills to 100% causing filesystem errors; write failures crash database.

Solution: Alert at 75%, expand at 80%. Size initial PVC at 130% of expected data volume.

Anti-Pattern: Using HostPath volumes in production

Problem: Data tied to specific node; Pod mobility lost; no redundancy; security risk (host filesystem access).

Solution: Use PVCs for all persistent data. HostPath only for system-level DaemonSet pods (log collection, etc.).

Anti-Pattern: emptyDir for persistent application data

Problem: emptyDir is ephemeral; data lost on Pod restart or node failure. Applications with state must use PVCs.

Solution: Use emptyDir for temp files, caches, shared memory between containers. Never for business data.

Anti-Pattern: Not testing restore procedures

Problem: Backups exist but restore has never been tested. Discover restore is broken during actual disaster.

Solution: Quarterly restore drills. Automate restore tests as CronJob to staging namespace. Measure RTO.

Anti-Pattern: Single StorageClass for all workloads

Problem: Databases sharing storage class with logs; noisy workloads impact database I/O.

Solution: Define tiered StorageClasses: ultra-ssd (databases), fast-ssd (app storage), standard (logs/batch).

CHAPTER 17

Hands-On Exercises

Exercise 7.1 -- Dynamic Provisioning and Expansion

Experience the full dynamic provisioning lifecycle:

```
# 1. Create PVC (triggers dynamic provisioning): kubectl apply -f - <<'YAML' apiVersion: v1
kind: PersistentVolumeClaim metadata: name: test-pvc spec: accessModes: [ReadWriteOnce]
storageClassName: standard resources: requests: storage: 5Gi YAML kubectl get pvc test-pvc -w
# Watch: Pending -> Bound (when Pod is created for WaitForFirstConsumer) # 2. Write data and
verify persistence: kubectl run writer --image=busybox --restart=Never \ --command -- sh -c
'echo PERSISTENT > /data/test.txt; sleep 3600' \ --overrides='{"spec":{"volumes":[{"name":"d",
"persistentVolumeClaim":{"claimName":"test-pvc"}}],"containers":[{"name":"writer","volumeMounts":[{"name":"d","mountPath":"/data"}]}}}' # 3. Expand PVC (edit storage request): kubectl
patch pvc test-pvc -p '{"spec":{"resources":{"requests":{"storage":"10Gi"}}}}' kubectl get pvc
test-pvc
```

Exercise 7.2 -- Volume Snapshot and Restore

Create a snapshot and restore from it:

```
# Install snapshot CRDs (if not present): kubectl apply -f https://raw.githubusercontent.com/k
ubernetes-csi/external-snapshotter/master/client/config/crd/snapshot.storage.k8s.io_volumesnap
shotclasses.yaml # Create VolumeSnapshot: kubectl apply -f - <<'YAML' apiVersion:
snapshot.storage.k8s.io/v1 kind: VolumeSnapshot metadata: name: test-snap spec:
volumeSnapshotClassName: csi-snapclass source: persistentVolumeClaimName: test-pvc YAML # Wait
for snapshot to be ready: kubectl get volumesnapshot test-snap -w # Restore into new PVC:
kubectl apply -f - <<'YAML' apiVersion: v1 kind: PersistentVolumeClaim metadata: name:
test-restored spec: dataSource: name: test-snap kind: VolumeSnapshot apiGroup:
snapshot.storage.k8s.io accessModes: [ReadWriteOnce] resources: requests: storage: 5Gi YAML
```

End of Part VII -- Continue to Part VIII: Security

Part VIII delivers the comprehensive enterprise security handbook: Zero Trust architecture on Kubernetes, SPIFFE/SPIRE workload identity, HashiCorp Vault integration, External Secrets Operator, certificate lifecycle with cert-manager, OPA Gatekeeper and Kyverno policy-as-code, Pod Security Standards enforcement, runtime security with Falco and Tetragon, Confidential Containers and Confidential Computing, and compliance mapping to NIST, CIS Benchmarks, ISO 27001, SOC 2, PCI DSS, HIPAA, DORA, and the EU AI Act.